

# HTML Tutorial

---

## A Complete Guide

11 chapters compiled into one PDF

---

**Snehal Kadiya**

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

## Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

**Please do not print this document unnecessarily.** Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/html/>

# Getting Started with HTML

---

## What is HTML?

HTML stands for **HyperText Markup Language**. It is the standard language used to create and structure content on the web. Every website you visit is built using HTML (along with CSS and JavaScript).

**Think of HTML as the skeleton of a webpage:** it gives structure to your content — headings, paragraphs, images, links, and more.

## Your First HTML Page

An HTML file is just a plain text file with a `.html` extension. Here's the simplest possible HTML page:

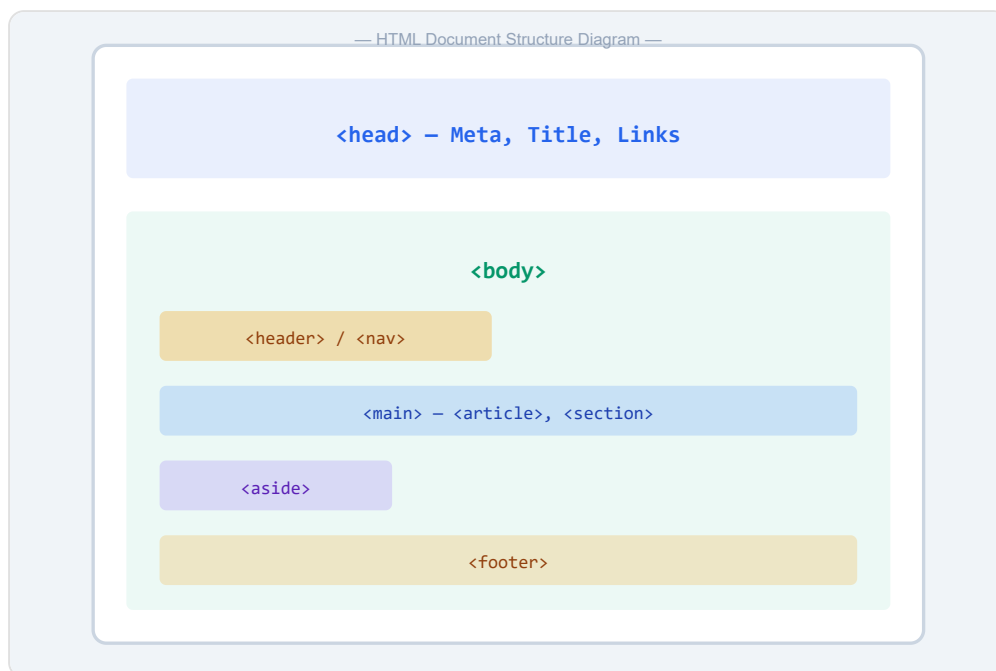
```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first HTML page.</p>
  </body>
</html>
```

## HTML Document Structure

Let's break down what each part means:

- `<!DOCTYPE html>` — tells the browser this is an HTML5 document

- `<html>` — the root element that wraps all content
- `<head>` — contains meta-information (title, links, styles)
- `<body>` — contains the visible content of the page



## Tools You Need

- **A text editor:** VS Code (recommended), Notepad++, or any basic text editor
- **A web browser:** Chrome, Firefox, Edge, or Safari

**Pro tip:** In VS Code, type `!` and press Tab to auto-generate a basic HTML skeleton.

## HTML Tags & Elements

HTML uses **tags** to mark up content. Most tags come in pairs — an opening tag and a closing tag:

```
<tagname>content goes here</tagname>
```

Some tags are self-closing (void elements):

```
<br>    <hr>    
```

## Understanding Attributes

Tags can have **attributes** that provide additional information:

```
<a href="https://example.com">Visit Example</a>

```

Here href, src, and alt are attributes.

## Comments in HTML

Comments help you document your code and are ignored by the browser:

```
<!-- This is a comment -->
<!-- TODO: Add navigation menu here -->
```

### Practice Task

Create a new file called `my-first-page.html`. Write a complete HTML document that includes:

- A proper `<!DOCTYPE html>` declaration
- A descriptive `<title>`
- At least one `<h1>` heading
- Two paragraphs of text (`<p>`)
- An HTML comment

Open it in your browser to see the result!

# Text & Structure

---

## Headings

HTML provides six levels of headings, from `<h1>` (most important) to `<h6>` (least important):

```
<h1>Main Title</h1>
<h2>Section Heading</h2>
<h3>Sub-section Heading</h3>
<h4>Detailed Sub-section</h4>
<h5>Minor Heading</h5>
<h6>Smallest Heading</h6>
```

**SEO Tip:** Use only one `<h1>` per page, and nest headings hierarchically — don't skip levels.

## Paragraphs

Use the `<p>` tag for paragraphs. Browsers automatically add space before and after each paragraph:

```
<p>This is a paragraph of text. It can contain several sentences and will
<p>This is another paragraph.</p>
```

## Text Formatting

HTML offers many inline elements for styling text:

```
<strong>Bold text</strong>      <!-- Important text -->
<em>Italic text</em>           <!-- Emphasized text -->
<u>Underlined text</u>
<mark>Highlighted text</mark>
<small>Smaller text</small>
<del>Deleted text</del>
<ins>Inserted text</ins>
<sup>Superscript</sup>          <!-- e.g., E=mc2 -->
<sub>Subscript</sub>           <!-- e.g., H2O -->
```

Output: **Bold**, *Italic*, Underlined, **Marked**, small, ~~deleted~~, inserted, E=mc<sup>2</sup>, H<sub>2</sub>O.

## Lists

### Unordered Lists

```
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Oranges</li>
</ul>
```

Renders as a bullet-point list.

### Ordered Lists

```
<ol>
  <li>First step</li>
  <li>Second step</li>
  <li>Third step</li>
</ol>
```

Renders as a numbered list.

## Definition Lists

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

## Quotations

```
<blockquote cite="https://example.com">
  This is a long block quotation.
</blockquote>

The philosopher said, <q>Cogito ergo sum</q>.
```

## Semantic HTML for Structure

Semantic tags clearly describe their meaning to both the browser and developers:

```
<header>  <!-- Site or section header -->
<nav>     <!-- Navigation links -->
<main>    <!-- Primary content -->
<article> <!-- Self-contained content -->
<section> <!-- Themed section -->
<aside>   <!-- Sidebar / related content -->
<footer>  <!-- Footer -->
```

`<head>` – Meta, Title, Links

`<body>`

`<header>` / `<nav>`

`<main>` – `<article>`, `<section>`

`<aside>`

`<footer>`

**Why semantic HTML matters:** It improves accessibility (screen readers rely on it), SEO, and code readability.

## Line Breaks & Horizontal Rules

```
<p>First line<br>Second line</p>  
<hr>  <!-- Thematic break -->
```

## Practice Task

Create an HTML page about your favorite hobby. Include:

- A main heading (<h1>)
- At least two sub-headings (<h2>)
- A paragraph for each section
- An unordered list and an ordered list
- Use <strong> and <em> in your text
- Wrap everything in semantic tags (<article>, <section>)

# Visuals & Design

---

## Images

The `<img>` tag embeds an image in your page. It is a self-closing tag with two required attributes:

```

```

- `src` — the image file path (URL or relative path)
- `alt` — alternative text for accessibility and when images fail to load

**Alt text best practice:** Describe the image content concisely. For decorative images, use `alt=""` (empty) so screen readers skip them.

## Responsive Images

Use the `srcset` attribute to serve different image sizes based on screen width:

```

```

## Figure & Figcaption

```
<figure>
  
  <figcaption>Figure 1: Login process flowchart</figcaption>
</figure>
```

## Linking Stylesheets

CSS is connected to HTML via the `<link>` tag inside `<head>`:

```
<link rel="stylesheet" href="styles.css">
```

## Inline Styles vs. Classes

CSS can be applied in three ways:

```
<!-- Inline style (avoid when possible) -->
<p style="color: blue; font-size: 18px;">Blue text</p>

<!-- Using a class (preferred) -->
<p class="highlight">Styled text</p>
```

## Colors in HTML

Colors can be specified in several formats:

```
<p style="color: red;">Named color</p>
<p style="color: #ff0000;">Hex code</p>
<p style="color: rgb(255, 0, 0);">RGB value</p>
<p style="color: hsl(0, 100%, 50%);">HSL value</p>
```

## Fonts & Typography

Use the `<link>` tag to include web fonts like Google Fonts:

```
<!-- In <head> -->
<link href="https://fonts.googleapis.com/css2?family=Open+Sans:wght@400;700" rel="stylesheet">

<!-- In CSS -->
body { font-family: 'Open Sans', sans-serif; }
```

# Icons

Icons can be added using icon libraries like Font Awesome or inline SVGs:

```
<!-- Font Awesome -->
<i class="fas fa-home"></i>

<!-- Inline SVG -->
<svg width="24" height="24" viewBox="0 0 24 24">
  <path d="M10 20v-6h4v6h5v-8h3L12 3 2 12h3v8z"/>
</svg>
```

## Practice Task

Create an HTML page that includes:

- An image with proper alt text wrapped in a `<figure>` with `<figcaption>`
- A paragraph with a colored background and styled font
- An icon using inline SVG or an icon library
- A `<link>` to an external stylesheet

# Layout & Organization

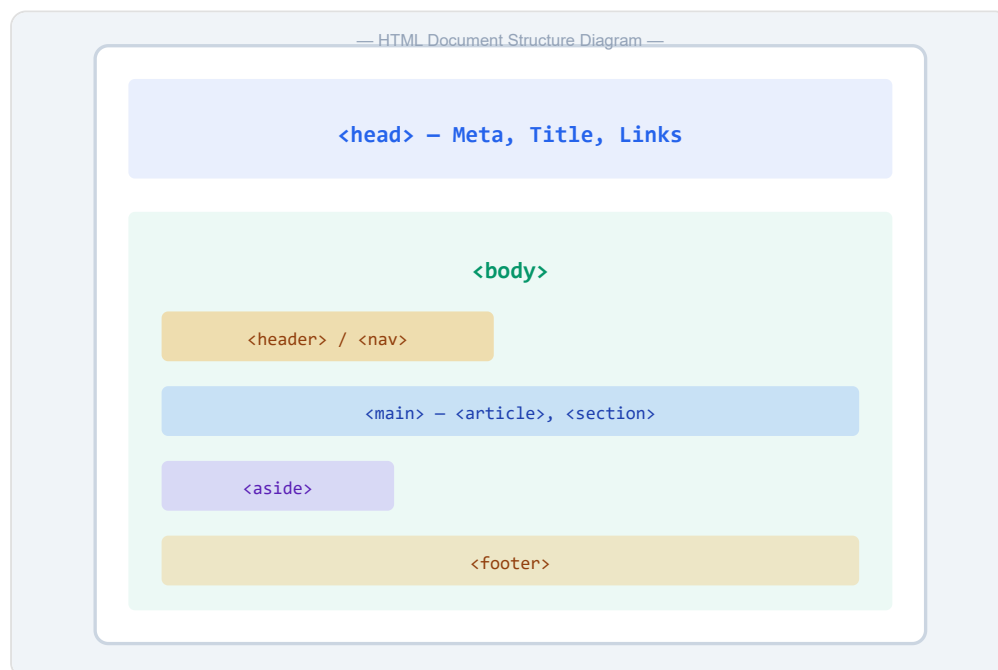
---

## The Box Model

Every HTML element is a rectangular box. The CSS box model consists of:

- **Content** — the actual text or image
- **Padding** — space between content and border
- **Border** — the edge around the element
- **Margin** — space outside the border

```
div {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid #333;  
  margin: 16px;  
}
```



## Display Properties

Elements can be displayed in different ways:

```
display: block;      /* Takes full width, starts new line (div, p, h1) */
display: inline;     /* Fits within text (span, a, strong) */
display: inline-block; /* Inline but can have width/height */
display: none;       /* Hidden, removed from layout */
```

## Flexbox Layout

Flexbox makes it easy to align items in rows or columns:

```
.container {
  display: flex;
  justify-content: center; /* horizontal alignment */
  align-items: center;    /* vertical alignment */
  gap: 16px;             /* space between items */
  flex-wrap: wrap;       /* wrap to next line */
}

.item {
  flex: 1;               /* grow equally */
  min-width: 200px;
}
```

## CSS Grid Layout

Grid creates two-dimensional layouts:

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr; /* three equal columns */
  gap: 20px;
}

.header { grid-column: 1 / -1; } /* span full width */
.sidebar { grid-column: 1 / 2; }
.main { grid-column: 2 / 4; }
```

## Responsive Design with Media Queries

Media queries adapt your layout to different screen sizes:

```
/* Default: mobile-first styles */
.container { width: 100%; padding: 10px; }

/* Tablet: 768px and up */
@media (min-width: 768px) {
  .container { width: 750px; margin: auto; }
}

/* Desktop: 1024px and up */
@media (min-width: 1024px) {
  .container { width: 960px; }
}
```

## Positioning

```
position: static; /* Default flow */
position: relative; /* Offset from normal position */
position: absolute; /* Relative to nearest positioned ancestor */
position: fixed; /* Relative to viewport (stays on scroll) */
position: sticky; /* Toggles between relative and fixed */
```

# Common Layout Patterns

## Holy Grail Layout

```
<header>Header</header>
<div class="middle">
  <nav>Left sidebar</nav>
  <main>Main content</main>
  <aside>Right sidebar</aside>
</div>
<footer>Footer</footer>
```

### Practice Task

Create a responsive page layout with:

- A header with a site title
- A navigation bar with 3 links (use Flexbox)
- A main content area and a sidebar (use CSS Grid)
- A footer
- Make it responsive: stack vertically on mobile (media query at 768px)

# Advanced Concepts

---

## Meta Tags

Meta tags provide metadata about the HTML document. They live inside the `<head>`:

```
<meta charset="UTF-8">
<meta name="description" content="Free HTML tutorial for beginners">
<meta name="keywords" content="HTML, tutorial, beginner, web development">
<meta name="author" content="Your Name">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Open Graph (for social media sharing) -->
<meta property="og:title" content="HTML Tutorial">
<meta property="og:description" content="Learn HTML from scratch">
<meta property="og:image" content="images/og-preview.png">
<meta property="og:url" content="https://example.com/html-tutorial">

<!-- Twitter Card -->
<meta name="twitter:card" content="summary_large_image">
```

## SEO (Search Engine Optimization)

Improve your site's visibility on search engines:

- Use semantic HTML tags (`<article>`, `<nav>`, `<header>`)
- Write unique `<title>` and `<meta name="description">` for each page
- Use heading tags hierarchically (only one `<h1>` per page)
- Add descriptive alt text to images
- Create a `sitemap.xml` and submit it to search engines

## ARIA & Accessibility

ARIA (Accessible Rich Internet Applications) attributes help assistive technology understand your page:

```
<nav aria-label="Main navigation">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/about">About</a></li>
  </ul>
</nav>

<button aria-label="Close dialog" onclick="closeModal()">
  &times;
</button>

<div role="alert" aria-live="polite">
  Form submitted successfully!
</div>
```

## Microdata & Schema.org

Microdata helps search engines understand your content better:

```
<article itemscope itemtype="https://schema.org/Article">
  <h1 itemprop="headline">How to Learn HTML</h1>
  <p itemprop="description">A beginner's guide to HTML.</p>
  <span itemprop="author" itemscope itemtype="https://schema.org/Person">
    <span itemprop="name">John Doe</span>
  </span>
  <time itemprop="datePublished" datetime="2026-01-15">Jan 15, 2026</time>
</article>
```

## Data Attributes

Custom data-\* attributes store extra information in HTML elements:

```
<button data-user-id="123" data-role="admin" onclick="showUser(this)">
  User Profile
</button>

<script>
  function showUser(btn) {
    const userId = btn.dataset.userId;
    const role = btn.dataset.role;
    alert(`User ${userId} is an ${role}`);
  }
</script>
```

## HTML Templates

The `<template>` tag holds HTML that isn't rendered until JavaScript clones it:

```
<template id="card-template">
  <div class="card">
    <img src="" alt="">
    <h3></h3>
    <p></p>
  </div>
</template>

<script>
  const clone = document.getElementById('card-template').content.cloneNode()
  clone.querySelector('h3').textContent = 'Dynamic Title';
  document.body.appendChild(clone);
</script>
```

## Practice Task

Take your hobby page from Chapter 2 and enhance it with:

- Proper meta tags (viewport, description, Open Graph)
- ARIA labels on navigation links
- A data-\* attribute on an element
- Schema.org microdata for an article

# Forms & Inputs

---

## The Form Element

Forms are used to collect user input and send it to a server:

```
<form action="/submit" method="POST">
  <!-- form elements go here -->
  <button type="submit">Submit</button>
</form>
```

### Form Attributes

- **action** — URL where the form data is sent
- **method** — HTTP method: GET or POST
- **name** — identifies the form
- **novalidate** — disables browser validation

## Input Types

HTML5 offers many input types for different kinds of data:

```
<input type="text" name="username">
<input type="email" name="email">
<input type="password" name="password">
<input type="number" name="age" min="0" max="150">
<input type="tel" name="phone">
<input type="url" name="website">
<input type="date" name="birthday">
<input type="time" name="appointment">
<input type="color" name="favorite-color">
<input type="range" name="volume" min="0" max="100">
<input type="file" name="document" accept=".pdf,.doc">
<input type="checkbox" name="subscribe" value="newsletter">
<input type="radio" name="gender" value="male">
<input type="hidden" name="csrf-token" value="abc123">
```

## Labels & Accessibility

Always associate a `<label>` with each input:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" required>

<!-- Or wrap input inside label -->
<label>
  <input type="checkbox" name="agree">
  I agree to the terms
</label>
```

## Textareas & Selects

```
<textarea name="message" rows="5" cols="40" placeholder="Your message...">

<select name="country">
  <option value="">Select a country</option>
  <option value="in">India</option>
  <option value="us">United States</option>
  <option value="uk">United Kingdom</option>
</select>
```

## Fieldset & Legend

Group related fields together:

```
<fieldset>
  <legend>Personal Information</legend>
  <label for="fname">First Name:</label>
  <input type="text" id="fname" name="fname">
  <label for="lname">Last Name:</label>
  <input type="text" id="lname" name="lname">
</fieldset>
```

## HTML5 Form Validation

|                                                  |                              |
|--------------------------------------------------|------------------------------|
| <input type="text" required>                     | <!-- Required field -->      |
| <input type="email">                             | <!-- Must be valid email --> |
| <input type="text" minlength="3" maxlength="20"> | <!-- Length limits -->       |
| <input type="number" min="1" max="10">           | <!-- Range limits -->        |
| <input type="text" pattern="[A-Za-z]+">          | <!-- Regex pattern -->       |

**Custom validation messages:** Use JavaScript's `setCustomValidity()` to override default messages.

## Complete Form Example

```
<form action="/register" method="POST">
  <fieldset>
    <legend>Create Account</legend>

    <label for="name">Full Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label for="pass">Password (min 8 chars):</label>
    <input type="password" id="pass" name="pass" minlength="8" required>

    <label>
      <input type="checkbox" name="terms" required>
      I accept the Terms & Conditions
    </label>

    <button type="submit">Register</button>
  </fieldset>
</form>
```

### Practice Task

Create a feedback form with:

- Name, email, and phone number fields with proper types
- A dropdown to select a topic (select with 4+ options)
- A textarea for the message
- A rating scale using radio buttons or a range slider
- Required field validation
- All inputs properly labeled for accessibility

# Graphics

---

## SVG (Scalable Vector Graphics)

SVG is an XML-based format for vector graphics. Unlike raster images (JPEG, PNG), SVGs scale infinitely without losing quality.

### Basic Shapes

```
<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <!-- Rectangle -->
  <rect x="10" y="10" width="80" height="50" fill="blue" stroke="black" st

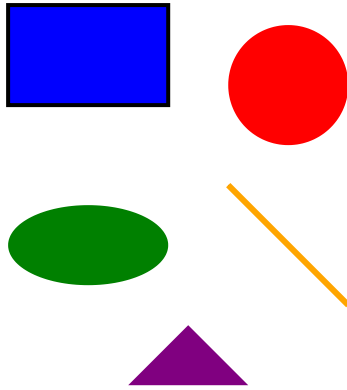
  <!-- Circle -->
  <circle cx="150" cy="50" r="30" fill="red"/>

  <!-- Ellipse -->
  <ellipse cx="50" cy="130" rx="40" ry="20" fill="green"/>

  <!-- Line -->
  <line x1="120" y1="100" x2="180" y2="160" stroke="orange" stroke-width="

  <!-- Polygon (triangle) -->
  <polygon points="100,170 70,200 130,200" fill="purple"/>
</svg>
```

Rendered result:



## Text in SVG

```
<svg width="300" height="100" xmlns="http://www.w3.org/2000/svg">
  <text x="10" y="40" font-family="Arial" font-size="24" fill="navy" font-
    Hello, SVG!
  </text>
  <text x="10" y="70" font-family="Arial" font-size="14" fill="gray">
    Scalable Vector Graphics
  </text>
</svg>
```

## Canvas

The `<canvas>` element provides a pixel-based drawing surface controlled by JavaScript:

```
<canvas id="myCanvas" width="400" height="250" style="border:1px solid #ccc">

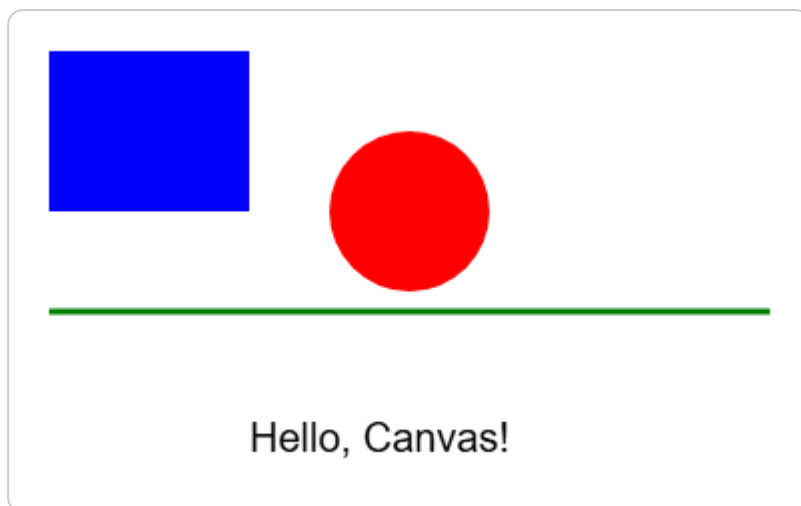
<script>
  const canvas = document.getElementById('myCanvas');
  const ctx = canvas.getContext('2d');

  // Rectangle
  ctx.fillStyle = 'blue';
  ctx.fillRect(20, 20, 100, 80);

  // Circle
  ctx.beginPath();
  ctx.arc(200, 100, 40, 0, Math.PI * 2);
  ctx.fillStyle = 'red';
  ctx.fill();

  // Line
  ctx.beginPath();
  ctx.moveTo(20, 150);
  ctx.lineTo(380, 150);
  ctx.strokeStyle = 'green';
  ctx.lineWidth = 3;
  ctx.stroke();

  // Text
  ctx.font = '20px Arial';
  ctx.fillStyle = 'black';
  ctx.fillText('Hello, Canvas!', 120, 220);
</script>
```



## SVG vs Canvas

| SVG                                     | Canvas                                  |
|-----------------------------------------|-----------------------------------------|
| Vector-based (resolution independent)   | Pixel-based (resolution dependent)      |
| Each element is a DOM node              | Single bitmap surface                   |
| Better for static, interactive graphics | Better for animations, games, real-time |
| Accessible (elements can have ARIA)     | Not accessible by default               |

### Practice Task

Create an HTML page that draws:

- An SVG with a star shape (use `<polygon>`) and colored text
- A Canvas drawing with a gradient rectangle and a smiley face circle

# Media

---

## Audio

The `<audio>` element embeds sound files:

```
<audio controls>
  <source src="audio/song.mp3" type="audio/mpeg">
  <source src="audio/song.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

### Audio Attributes

- `controls` — shows play/pause/volume controls
- `autoplay` — starts playing automatically (use with caution)
- `loop` — repeats the audio
- `muted` — starts muted
- `preload` — hints at what to load: none, metadata, auto

**Supported formats:** MP3, WAV, and OGG. Always provide multiple source formats for browser compatibility.

## Video

The `<video>` element embeds video files:

```
<video width="640" height="360" controls poster="images/thumbnail.jpg">
  <source src="video/tutorial.mp4" type="video/mp4">
  <source src="video/tutorial.webm" type="video/webm">
  <track src="subtitles-en.vtt" kind="subtitles" srclang="en" label="English">
  Your browser does not support the video element.
</video>
```

## Video Attributes

- `controls` — playback controls
- `autoplay` — start automatically
- `loop` — repeat
- `muted` — start muted (required for autoplay in most browsers)
- `poster` — thumbnail image before playback
- `width / height` — dimensions
- `playsinline` — play inline on mobile (no fullscreen)

## Subtitles & Captions

Use the `<track>` element to add subtitles, captions, or descriptions:

```
<track src="captions-en.vtt" kind="captions" srclang="en" label="English">
<track src="captions-es.vtt" kind="captions" srclang="es" label="Spanish">
<track src="descriptions.vtt" kind="descriptions" srclang="en" label="English">
```

The `.vtt` file format (WebVTT):

WEBVTT

00:00:01.000 --> 00:00:05.000

Welcome to this tutorial.

00:00:06.000 --> 00:00:10.000

Today we'll learn about HTML media elements.

# Embedded Content

## iframes

Embed another page or external content:

```
<iframe src="https://example.com" width="100%" height="400" style="border:
allowfullscreen loading="lazy" title="Example page"></iframe>
```

**Security tip:** Use the sandbox attribute to restrict what an iframe can do:  
`sandbox="allow-scripts allow-same-origin"`

## YouTube Videos

```
<iframe width="560" height="315"
src="https://www.youtube.com/embed/VIDEO_ID"
title="YouTube video"
frameborder="0"
allow="accelerometer; autoplay; clipboard-write; encrypted-media;
allowfullscreen"></iframe>
```

## Best Practices for Media

- Always provide **fallback content** for unsupported formats
- Use **multiple source formats** for cross-browser support
- Set width and height to prevent layout shifts
- Use `loading="lazy"` on iframes for performance
- Add **captions/subtitles** for accessibility
- Compress audio and video files for faster loading

## Practice Task

Create an HTML page that includes:

- An audio player with controls (use a sample audio URL or a local file)
- A video player with a poster image and controls
- An embedded YouTube video using an iframe
- A Google Maps embed showing your city

# APIs & Advanced Features

## Geolocation API

Access the user's geographic location (with permission):

```
<button onclick="getLocation()">Show My Location</button>
<p id="location-output"></p>

<script>
  function getLocation() {
    if (!navigator.geolocation) {
      document.getElementById('location-output').textContent =
        'Geolocation is not supported by your browser.';
      return;
    }

    navigator.geolocation.getCurrentPosition(
      function(position) {
        const { latitude, longitude } = position.coords;
        document.getElementById('location-output').innerHTML =
          `Latitude: ${latitude}<br>Longitude: ${longitude}`;
      },
      function(error) {
        document.getElementById('location-output').textContent =
          'Error: ' + error.message;
      }
    );
  }
</script>
```

**Privacy:** Browsers always ask for user permission before sharing location data.

## Drag and Drop API

Make elements draggable using native HTML5 drag-and-drop:

```
<div id="drag-source" draggable="true" ondragstart="drag(event)" style="width: 100px; height: 30px; border: 1px solid black; background-color: #f0f0f0; text-align: center; line-height: 30px;">
  Drag me
</div>

<div id="drop-zone" ondrop="drop(event)" ondragover="allowDrop(event)" style="width: 100px; height: 30px; border: 1px solid black; background-color: #f0f0f0; text-align: center; line-height: 30px;">
  Drop here
</div>

<script>
  function allowDrop(e) { e.preventDefault(); }
  function drag(e) { e.dataTransfer.setData('text', e.target.id); }
  function drop(e) {
    e.preventDefault();
    const data = e.dataTransfer.getData('text');
    e.target.appendChild(document.getElementById(data));
  }
</script>
```

## Web Storage API

Store data in the browser with `localStorage` (persists) and `sessionStorage` (per tab):

```

<!-- Saving user preferences -->
<input type="text" id="username" placeholder="Enter your name">
<button onclick="saveName()">Save</button>
<p id="greeting"></p>

<script>
  // Load saved data on page load
  window.onload = function() {
    const name = localStorage.getItem('username');
    if (name) {
      document.getElementById('greeting').textContent = 'Hello, ' + name + '
      document.getElementById('username').value = name;
    }
  };

  function saveName() {
    const name = document.getElementById('username').value;
    localStorage.setItem('username', name);
    document.getElementById('greeting').textContent = 'Hello, ' + name + '
  }

  // sessionStorage works the same but clears when tab closes
  // sessionStorage.setItem('key', 'value');
  // sessionStorage.getItem('key');
  // sessionStorage.removeItem('key');
  // sessionStorage.clear();
</script>

```

## History API

Manipulate the browser's session history without page reloads:

```
<button onclick="pushState()">Push State</button>
<button onclick="goBack()">Go Back</button>

<script>
  function pushState() {
    history.pushState({ page: 2 }, 'Page 2', '?page=2');
    document.title = 'Page 2';
  }

  window.onpopstate = function(e) {
    alert('You navigated back! State: ' + JSON.stringify(e.state));
  };

  function goBack() {
    history.back();
  }
</script>
```

## Fetch API

Make HTTP requests to fetch data from APIs:

```
<button onclick="loadData()">Load Users</button>
<ul id="user-list"></ul>

<script>
  async function loadData() {
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/u
      if (!response.ok) throw new Error('Network error');
      const users = await response.json();
      const list = document.getElementById('user-list');
      list.innerHTML = users.slice(0, 5).map(user =>
        `<li>${user.name} (${user.email})</li>`
      ).join('');
    } catch (error) {
      alert('Failed to load data: ' + error.message);
    }
  }
</script>
```

## Notifications API

```
<button onclick="showNotification()">Show Notification</button>

<script>
  function showNotification() {
    if (!('Notification' in window)) {
      alert('Notifications not supported.');
```

```
      return;
    }

    if (Notification.permission === 'granted') {
      new Notification('Hello!', {
        body: 'This is a notification from your HTML page.',
        icon: 'images/icon.png'
      });
    } else if (Notification.permission !== 'denied') {
      Notification.requestPermission().then(permission => {
        if (permission === 'granted') showNotification();
      });
    }
  }
</script>
```

### Practice Task

Build a simple "Notes App" page that:

- Has a text input and "Add Note" button
- Saves notes using `localStorage`
- Displays all saved notes in a list
- Has a "Clear All" button
- Loads saved notes automatically on page load

# Practice & Certification

---

## HTML Practice Projects

Build these projects to reinforce your skills, from beginner to advanced:

### Beginner Projects

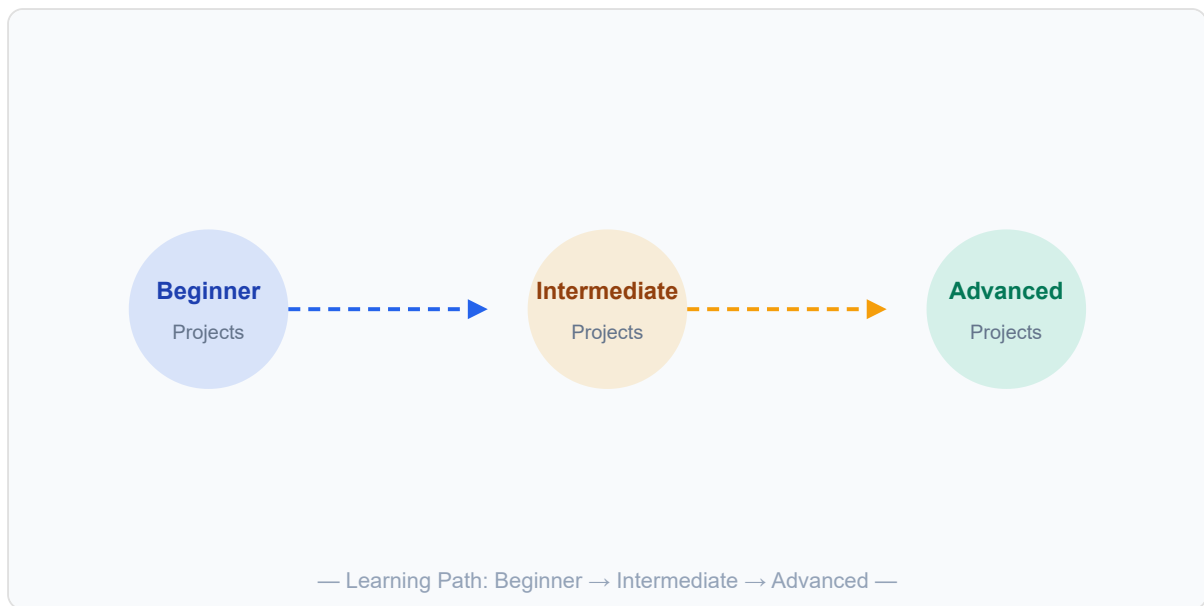
1. **Personal Profile Page** — Create a single-page profile with your photo, bio, skills list, and social media links.
2. **Recipe Page** — A recipe with ingredients (unordered list), steps (ordered list), and an image.
3. **Photo Gallery** — A grid of images with captions using `<figure>` and `<figcaption>`.

### Intermediate Projects

1. **Multi-Page Website** — Create a 4-page website (Home, About, Services, Contact) with a shared navigation bar and consistent styling.
2. **Registration Form** — Build a complete registration form with validation using various input types.
3. **Blog Layout** — A blog homepage with article cards, a sidebar with categories, and a footer.

### Advanced Projects

1. **Portfolio Website** — Your personal portfolio with a hero section, project showcase, testimonials, and contact form.
2. **E-Commerce Product Page** — A product detail page with image gallery, size/color selectors, and add-to-cart functionality.
3. **Interactive Dashboard** — A dashboard layout with charts (Canvas/SVG), data tables, and filter controls.



## Certification Paths

Earning a certification validates your skills to employers. Here are recommended paths:

### Free Certifications

- **freeCodeCamp** — Responsive Web Design Certification (300 hours, free)
- **Sololearn** — HTML Course with certificate
- **W3Schools** — HTML Certificate

### Paid Certifications

- **Microsoft Learn** — HTML/CSS learning paths with badges
- **LinkedIn Learning** — Become a Front-End Web Developer path
- **Google Digital Garage** — Fundamentals of Digital Marketing

## Sample Practice Test

Test your HTML knowledge with these questions:

1. What does HTML stand for?
2. Which tag is used for the largest heading?
3. What attribute specifies alternative text for an image?

4. Which tag creates an unordered list?
5. What is the correct HTML element for inserting a line break?
6. Which input type creates a checkbox?
7. What does the `alt` attribute do?
8. Which element is used to define navigation links?
9. How do you add a comment in HTML?
10. What is the purpose of the `<meta charset="UTF-8">` tag?

Answers: 1. HyperText Markup Language, 2. `<h1>`, 3. `alt`, 4. `<ul>`, 5. `<br>`, 6. `<input type="checkbox">`, 7. Alternative text for accessibility, 8. `<nav>`, 9. `<!-- comment -->`, 10. Sets character encoding for the document.

## Next Steps After HTML

- **CSS** — Learn styling, layouts (Flexbox, Grid), animations, and responsive design
- **JavaScript** — Add interactivity, DOM manipulation, event handling, and API calls
- **Version Control (Git)** — Track changes, collaborate, and deploy your sites
- **Build Tools** — Explore npm, Webpack, Vite for modern development workflows
- **Frameworks** — React, Vue, or Angular for building complex web applications

### Final Project Challenge

Combine everything you've learned — build a complete **personal portfolio website** with:

- A responsive layout (Flexbox/Grid + media queries)
- Semantic HTML5 structure
- Optimized images with proper alt text
- A contact form with validation
- Embedded media (video or map)
- Accessibility features (ARIA labels, skip links)
- meta tags for SEO and social sharing

# References

---

## HTML Tag Quick Reference

Here is a comprehensive list of HTML tags organized by category:

### Document Structure

```
<!DOCTYPE html>    <!-- Document type declaration -->
<html>              <!-- Root element -->
<head>              <!-- Meta-information -->
<title>             <!-- Page title (required) -->
<body>              <!-- Visible content -->
```

## Text & Content

|                                       |                                                      |
|---------------------------------------|------------------------------------------------------|
| <code>&lt;h1&gt;...&lt;/h6&gt;</code> | <code>&lt;!-- Headings level 1-6 --&gt;</code>       |
| <code>&lt;p&gt;</code>                | <code>&lt;!-- Paragraph --&gt;</code>                |
| <code>&lt;br&gt;</code>               | <code>&lt;!-- Line break --&gt;</code>               |
| <code>&lt;hr&gt;</code>               | <code>&lt;!-- Horizontal rule --&gt;</code>          |
| <code>&lt;strong&gt;</code>           | <code>&lt;!-- Important text (bold) --&gt;</code>    |
| <code>&lt;em&gt;</code>               | <code>&lt;!-- Emphasized text (italic) --&gt;</code> |
| <code>&lt;mark&gt;</code>             | <code>&lt;!-- Marked/highlighted text --&gt;</code>  |
| <code>&lt;small&gt;</code>            | <code>&lt;!-- Smaller text --&gt;</code>             |
| <code>&lt;del&gt;</code>              | <code>&lt;!-- Deleted text --&gt;</code>             |
| <code>&lt;ins&gt;</code>              | <code>&lt;!-- Inserted text --&gt;</code>            |
| <code>&lt;sub&gt;</code>              | <code>&lt;!-- Subscript --&gt;</code>                |
| <code>&lt;sup&gt;</code>              | <code>&lt;!-- Superscript --&gt;</code>              |
| <code>&lt;blockquote&gt;</code>       | <code>&lt;!-- Block quotation --&gt;</code>          |
| <code>&lt;q&gt;</code>                | <code>&lt;!-- Inline quotation --&gt;</code>         |
| <code>&lt;abbr&gt;</code>             | <code>&lt;!-- Abbreviation --&gt;</code>             |
| <code>&lt;address&gt;</code>          | <code>&lt;!-- Contact information --&gt;</code>      |
| <code>&lt;cite&gt;</code>             | <code>&lt;!-- Citation --&gt;</code>                 |
| <code>&lt;code&gt;</code>             | <code>&lt;!-- Code fragment --&gt;</code>            |
| <code>&lt;pre&gt;</code>              | <code>&lt;!-- Preformatted text --&gt;</code>        |

## Lists

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| <code>&lt;ul&gt;</code> | <code>&lt;!-- Unordered list --&gt;</code>         |
| <code>&lt;ol&gt;</code> | <code>&lt;!-- Ordered list --&gt;</code>           |
| <code>&lt;li&gt;</code> | <code>&lt;!-- List item --&gt;</code>              |
| <code>&lt;dl&gt;</code> | <code>&lt;!-- Definition list --&gt;</code>        |
| <code>&lt;dt&gt;</code> | <code>&lt;!-- Definition term --&gt;</code>        |
| <code>&lt;dd&gt;</code> | <code>&lt;!-- Definition description --&gt;</code> |

## Links & Images

```
<a href="">          <!-- Hyperlink -->
<img src="" alt=""> <!-- Image -->
<figure>            <!-- Figure container -->
<figcaption>        <!-- Figure caption -->
<map>               <!-- Image map -->
<area>              <!-- Clickable area on map -->
```

## Tables

```
<table>             <!-- Table -->
<caption>           <!-- Table caption -->
<thead>             <!-- Table header group -->
<tbody>             <!-- Table body group -->
<tfoot>             <!-- Table footer group -->
<tr>               <!-- Table row -->
<th>               <!-- Table header cell -->
<td>               <!-- Table data cell -->
<colgroup>         <!-- Column group -->
<col>              <!-- Column properties -->
```

## Forms

```
<form>              <!-- Form container -->
<input>             <!-- Input field -->
<textarea>          <!-- Multi-line text input -->
<select>            <!-- Dropdown list -->
<option>            <!-- Option in dropdown -->
<optgroup>          <!-- Option group -->
<label>             <!-- Input label -->
<fieldset>          <!-- Field group -->
<legend>            <!-- Fieldset caption -->
<button>            <!-- Clickable button -->
<datalist>          <!-- Autocomplete options -->
<output>            <!-- Calculation output -->
```

## Semantic & Layout

|                              |                                                      |
|------------------------------|------------------------------------------------------|
| <code>&lt;header&gt;</code>  | <code>&lt;!-- Page/section header --&gt;</code>      |
| <code>&lt;nav&gt;</code>     | <code>&lt;!-- Navigation --&gt;</code>               |
| <code>&lt;main&gt;</code>    | <code>&lt;!-- Main content --&gt;</code>             |
| <code>&lt;article&gt;</code> | <code>&lt;!-- Self-contained content --&gt;</code>   |
| <code>&lt;section&gt;</code> | <code>&lt;!-- Themed section --&gt;</code>           |
| <code>&lt;aside&gt;</code>   | <code>&lt;!-- Sidebar --&gt;</code>                  |
| <code>&lt;footer&gt;</code>  | <code>&lt;!-- Footer --&gt;</code>                   |
| <code>&lt;details&gt;</code> | <code>&lt;!-- Expandable details --&gt;</code>       |
| <code>&lt;summary&gt;</code> | <code>&lt;!-- Details summary --&gt;</code>          |
| <code>&lt;div&gt;</code>     | <code>&lt;!-- Generic block container --&gt;</code>  |
| <code>&lt;span&gt;</code>    | <code>&lt;!-- Generic inline container --&gt;</code> |

## Media & Embedded

|                             |                                                |
|-----------------------------|------------------------------------------------|
| <code>&lt;audio&gt;</code>  | <code>&lt;!-- Audio player --&gt;</code>       |
| <code>&lt;video&gt;</code>  | <code>&lt;!-- Video player --&gt;</code>       |
| <code>&lt;source&gt;</code> | <code>&lt;!-- Media source --&gt;</code>       |
| <code>&lt;track&gt;</code>  | <code>&lt;!-- Subtitles/captions --&gt;</code> |
| <code>&lt;iframe&gt;</code> | <code>&lt;!-- Inline frame --&gt;</code>       |
| <code>&lt;embed&gt;</code>  | <code>&lt;!-- Embedded content --&gt;</code>   |
| <code>&lt;object&gt;</code> | <code>&lt;!-- Embedded object --&gt;</code>    |
| <code>&lt;param&gt;</code>  | <code>&lt;!-- Object parameters --&gt;</code>  |
| <code>&lt;canvas&gt;</code> | <code>&lt;!-- Drawing canvas --&gt;</code>     |
| <code>&lt;svg&gt;</code>    | <code>&lt;!-- SVG graphics --&gt;</code>       |

## Meta & Scripting

|                               |                                                |
|-------------------------------|------------------------------------------------|
| <code>&lt;meta&gt;</code>     | <code>&lt;!-- Metadata --&gt;</code>           |
| <code>&lt;link&gt;</code>     | <code>&lt;!-- External resource --&gt;</code>  |
| <code>&lt;style&gt;</code>    | <code>&lt;!-- Embedded CSS --&gt;</code>       |
| <code>&lt;script&gt;</code>   | <code>&lt;!-- JavaScript --&gt;</code>         |
| <code>&lt;noscript&gt;</code> | <code>&lt;!-- Fallback for no JS --&gt;</code> |
| <code>&lt;base&gt;</code>     | <code>&lt;!-- Base URL --&gt;</code>           |
| <code>&lt;template&gt;</code> | <code>&lt;!-- HTML template --&gt;</code>      |
| <code>&lt;slot&gt;</code>     | <code>&lt;!-- Web component slot --&gt;</code> |

## HTML5 Input Types Summary

|                |        |          |        |
|----------------|--------|----------|--------|
| text           | email  | password | number |
| tel            | url    | date     | time   |
| datetime-local | month  | week     | color  |
| range          | file   | checkbox | radio  |
| hidden         | search | image    | submit |
| reset          | button |          |        |

## Global Attributes (usable on any element)

|                 |            |           |           |
|-----------------|------------|-----------|-----------|
| class           | id         | style     | title     |
| lang            | dir        | hidden    | tabindex  |
| role            | aria-*     | data-*    | draggable |
| contenteditable | spellcheck | translate |           |

## Useful Resources

- **MDN Web Docs** — [developer.mozilla.org/en-US/docs/Web/HTML](https://developer.mozilla.org/en-US/docs/Web/HTML)
- **W3Schools** — [w3schools.com/html/](https://w3schools.com/html/)
- **HTML Validator** — [validator.w3.org/](https://validator.w3.org/)
- **Can I Use** — Browser support tables at [caniuse.com/](https://caniuse.com/)
- **HTML Living Standard** — [html.spec.whatwg.org/](https://html.spec.whatwg.org/)

### Challenge

Build a "My HTML Cheat Sheet" page that you can reference in the future:

- Organize tags by category in separate sections
- Include a live example of each tag
- Add a search bar (just the HTML structure) to filter tags
- Style it with your own CSS design

**Keep learning!** HTML is the foundation of the web. Mastering it opens the door to front-end development, full-stack engineering, and beyond.